Java Collections Interview Handbook

Complete Revision Edition

Collections Framework

Why?

Arrays are fixed in size and one data structure cannot solve every problem efficiently. Java introduced the Collections Framework to provide specialized data structures for different use cases.

Hierarchy

List keeps order, Set keeps uniqueness, Queue processes elements, Map stores key-value pairs.

Interview Tip

Choose the collection based on the operation you perform most often, not by habit.

ArrayList

Why?

Dynamic array that grows automatically.

Internal Working

Backed by Object[]. When full, a larger array is created and all elements are copied.

Why get(index) is O(1)

Direct array indexing.

Why middle insertion is O(n)

Elements shift to make space.

Use When

Frequent reads, appending at end.

Avoid When

Frequent middle insertions/removals.

Remember

Dynamic Array → Fast Read → Slow Shift.

LinkedList

Why?

Designed for frequent insertions and deletions.

Internal Working

Doubly linked list. Each node stores previous, data and next.

Random Access

Traversal is required, therefore get(index) is O(n).

Insertion

Once the position is known only links change.

Remember

Train coach analogy: every coach knows previous and next.

HashMap

Why?

Avoid searching every element. Jump directly near the required data.

Internal Structure

Array(table[]) + Buckets + Nodes. Bucket simply means array index.

put() Flow

Key → hashCode() → Bucket Index → equals() if required → Insert or Update.

get() Flow

Key → hashCode() → Bucket → Traverse → equals() → Return value.

Collision

Different keys can reach the same bucket. Java stores multiple nodes there.

hashCode vs equals

hashCode decides bucket. equals decides exact key.

Custom Objects

Override both equals() and hashCode().

Capacity / Threshold

Default capacity 16, load factor 0.75, threshold 12.

Resize

Capacity doubles. Existing nodes are rehashed and may move to different buckets.

Treeification

Java 8+: bucket>=8 AND capacity>=64 converts linked list to Red-Black Tree.

Complexity

Average put/get/remove O(1). Worst case O(n), improved bucket lookup O(log n) after treeification.

Remember

HashMap = Array + Buckets + Nodes.

HashSet

Why?

Store only unique elements.

Internal Working

Internally uses HashMap. Element becomes key and PRESENT is stored as dummy value.

Duplicate Check

hashCode() → Bucket → equals() to confirm duplicate.

Remember

HashSet is HashMap without useful values.

Queue

Concept

FIFO.

Operations

offer, poll, peek. poll/peek return null on empty queue while remove/element throw exception.

Implementation

LinkedList is commonly used.

PriorityQueue

Concept

Priority decides removal order, not insertion order.

Internal Working

Binary Min Heap.

Complexity

peek O(1), offer/poll O(log n).

TreeMap

Why?

Need keys in sorted order.

Internal Working

Red-Black Tree.

Comparison

HashMap for speed, TreeMap for sorted keys.

Final Cheat Sheet

Choose

ArrayList=Fast Read, LinkedList=Frequent Modification, HashMap=Fast Lookup, HashSet=Unique Elements, Queue=FIFO, PriorityQueue=Priority, TreeMap=Sorted Keys

Interview Advice

Always explain WHY before stating complexity.